INSTRUMENT DOCUMENTATION

Status **Panel**

Operator manual · Galactic Unicorn 53×11

Twelve screens, four buttons that matter, one alarm, and the rule for every colour on the wall.

The panel is a monitoring instrument, not a decoration. It renders fleet state that xenon computed, over a link it does not control, on a matrix with 583 pixels and no room for a second reading of anything. This manual states what each region means, so that a glance from across the room is a measurement rather than an impression.

Every layout, threshold and colour here was read out of the shipped firmware renderers and the build contracts. Where the firmware deviates from the design brief — and it does, in several places, for stated reasons — this manual documents the firmware, because the firmware is what is on the wall.

VERSION	1.1.0	HARDWARE	Pimoroni GU / RP2350	FORMAT	Letter · 816×1056
DATE	2026-08-17	HOST	lithium · WiFi (§21)	THEMES	Dark · Light

SECTION 00

Contents

Read sections 01–03 once. After that the manual is a lookup table: find the screen on the wall, read its page.

The screens changed on 5 November 2026. The current sixteen are in the *Panel Screen Guide* at akoria.net/docs/. This document describes the earlier screen set; its hardware, alarm and operations material still applies.

§	SECTION	WHAT IT SETTLES
01	The instrument	What is computing what, and where a wrong reading comes from
02	The physical panel	Nine buttons, brightness, sleep, and the one that acknowledges
03	Reading the matrix	Colour vocabulary, the two populations, quiet–healthy
04	A0 · State lattice	One LED per system, whole fleet, no glyphs
05	A1 · Flow gates	The live Ubiquiti path and the internet-inherits-router rule
06	A2 · MQ and SNMP	Four lanes, and what they actually carry
07	B0 · Condition counts	UP / DOWN / DEG / MNT and the tier flag word
08	B1 · Throughput	Utilisation bar, link percent, IN/OUT ticker
09	B2 · Load and latency	Mean load, the alternating P95/LOSS reading, the >85 count
10	C0 · Load distribution	35-bucket histogram plus the throughput ribbon
11	C1 · Live traces	Three stacked 53-sample histories
12	C2 · Pool load	Seven pools worst-first with a status key column
13	D0 · Reachability	The waterfall, one row per worst-first system
14	D1 · Pool bars	Proportional stacked pools and the alert ticker
15	D2 · Ambient field	The resting face, its sparks and its cycling readout
16	Alarms	Inlay, beacon, tone cadence, and what acknowledge does
17	The control page	Theme, screen, brightness, dwell, acknowledge, sleep
18	Rotation & weights	Per-screen enable, ¼×–10×, and who confirms a setting
19	The glance matrix	The UNO Q's own 13×8 summary of the panel
20	Troubleshooting	LINK LOST, NO DATA, stale, and nothing on the wall
21	Standalone WiFi	Life without lithium, and who wins when USB is plugged in
22	Power & battery	The ride-through cap, the battery glyph, the power-loss alarm

A convention used throughout. Screens are named `<theme><slot>` — A0 is theme A, slot 0. Matrix coordinates are given as x (column, 0–52, left to right) and y (row, 0–10, top to bottom). "Row 6" always means the seventh row down.

SECTION 01

The instrument

Three machines, one direction of travel. Knowing which link is which is how you tell a fleet fault from a panel fault without leaving the room.

XENON
Dashboard API. Computes the score, the alarm condition and the episode id. Serves `/api/panel`.

→

LITHIUM
The `solariPanel` daemon. Polls every 5 s, frames a snapshot to the panel every 2 s over USB, relays commands.

→

THE PANEL
RP2350 firmware. Animates at 25 Hz from the last snapshot it received, keeps its own history, owns the alarm.

What the panel decides for itself

The split matters, because it tells you where to look when a reading is wrong.

Computed on xenon, never on the panel

- **Score** and **alarm condition**. The firmware never recomputes either; it renders what arrived.
- **Episode id** — the identity of an alarm episode, and therefore what an acknowledge is scoped to.
- **Counts, pools, per-system state**, throughput, RTT and loss aggregates.
- **Alert subject and detail**, already uppercased and truncated to the panel's charset.

Owned by the panel

- **All history.** Ribbons, histograms and traces are accumulated from successive snapshots — history is never on the wire.
- **Rotation**: which screen is up, for how long, and which screens are skipped.
- **Brightness**, sleep, and the acknowledge state of the current episode.
- **The link verdict.** More than 15 s with no valid frame is LINK LOST, and that is a panel fault.

Two populations, deliberately

The panel counts two different things and never pretends they are one. Reading a "3" next to a headline of "9" is not a contradiction.

POPULATION	WHERE IT APPEARS	WHAT IT CONTAINS
`stateRoll`	B0's UP/DOWN/DEG/MNT, D2's "n SYS"	Agent nodes only — the same number the dashboard headline shows
`roster`	A0's lattice, pools, C2/D1 bars, B2's >85count	Nodes *plus* adopted probe-only assets; pool aggregates are computed over the full fleet, even beyond the 64 systems the wire carries

Staleness has two independent sources. The endpoint marks data stale when the newest monitoring sample is more than 30 s old; the daemon marks it stale when the payload it last fetched is more than 30 s old. Either one is enough. A stale flag does not blank the panel and does not raise an alarm — the screens keep animating on the last good data, which is why the flag exists at all.

SECTION 02

The physical panel

Nine buttons. Four of them choose a theme, four adjust brightness and volume, one sleeps the display — except while an alarm is armed, when every one of them does the same thing.

BUTTON	NORMAL ACTION	DETAIL
A	Theme ABSTRACT	Jumps to the theme's first *enabled* slot; rotation continues from there
B	Theme TEXT	As above
C	Theme CHARTS	As above
D	Theme INSTRUMENTS	As above. D1 is where the panel boots
LUX +	Brightness +5%	Latches **manual** — the light sensor stops driving brightness until AUTO is re-enabled from the control page
LUX −	Brightness −5%	Same latch. The driver floors at 25%, so the panel dims down but never out
VOL +	Help overlay	Two lines of help for the screen showing, for 30 s. VOL + again closes it; any other button closes it *and* acts. Rotation pauses while help is open
VOL −	Alarm tone quieter	Steps of 0.1, and the volume control; a reboot restores the default. *only* Affects the alarm triad only — the panel is otherwise silent
ZZZ	Sleep the display	Blanks the matrix. Any button wakes it. An arriving alarm wakes it once, on the rising edge

While an alarm is armed, every button is an acknowledge. Press A expecting a theme change and you will silence the alarm instead — the press is *consumed*, and does not also change theme, step brightness or toggle sleep. This is deliberate: at 3 a.m. the operator's first instinct is to hit something, and the panel treats that as intent to acknowledge rather than as intent to reconfigure.

Brightness

Automatic

The board's light sensor drives global brightness, smoothed, reaching full output at a raw reading of about 1600 — deliberately well below the sensor's ceiling, because the panel lives in direct daylight and that is the normal condition rather than the extreme.

Manual

Either LUX button, or the control page's slider, latches a manual level and stops the sensor from moving it. Only the page's AUTO button re-enables the sensor. The 25% driver floor applies in both modes, which is what keeps the alarm beacon visible at any setting.

Boot and sleep

Boot

The panel boots to **theme D, slot 1** — Pool bars, the "resting face": a proportional read of every pool plus the alert ticker. Before the first snapshot arrives it shows `NO DATA` rather than a black board.

Sleep

Sleep blanks the matrix but not the firmware: snapshots and history continue, and the alarm beacon is still painted. A sleeping panel with a fault shows four flashing red pixels top-right and nothing else — intended overnight behaviour, not a fault.

SECTION 03

Reading the matrix

583 LEDs, six state colours, and one rule that makes the wall legible: a healthy fleet is supposed to look almost dark.

STATE	TREATMENT	MEANING ON THE PANEL
Operational	`slate, dim, still`	Reporting and in tolerance. Brightness carries tier and load, not health
Degraded	`amber, slow pulse`	Reporting, one or more metrics over tolerance
Down	`red, bright`	Unreachable. A tier-0 system pulses faster than the rest
Maintenance	`violet, dashed`	Alerting suppressed by an operator. Reserved on the wire until the server grows the state
Unknown	`slate, dashed`	Enrolled but returning no telemetry. Dashed so it never reads as healthy
Azure	`structure, labels`	**Not a condition.** Chrome, labels, zero counts, and traffic that is merely present

Azure is never a status. The house rule is that a hue reporting condition may not also decorate the interface, and the panel obeys it: green, amber, red, violet and slate say something about the fleet; azure says "this is structure, or this is a figure with nothing wrong with it". A zero DOWN count renders azure and dim precisely so that a non-zero one, in red, is the only thing your eye finds.

Quiet-healthy

Nine systems in ten are fine at any moment. Painting the majority brightly is what hides the minority that is not, so the panel spends its brightness budget on faults. A wall that looks nearly dark is a wall reporting a healthy fleet — not a wall that has stopped working. Confirm the difference by watching for motion: every screen carries something that moves, and section 20 lists what to check when nothing does.

Three things brightness can mean

Tier

On A0, a system's brightness is scaled by its tier — tier 0 at full, then 0.62, 0.4 and 0.26. The core is literally brighter than the periphery.

Magnitude

On bars, ribbons and traces the brightest pixel is the tip — the current value. The body of the bar is dim so a row of bars reads as a profile.

Watermark

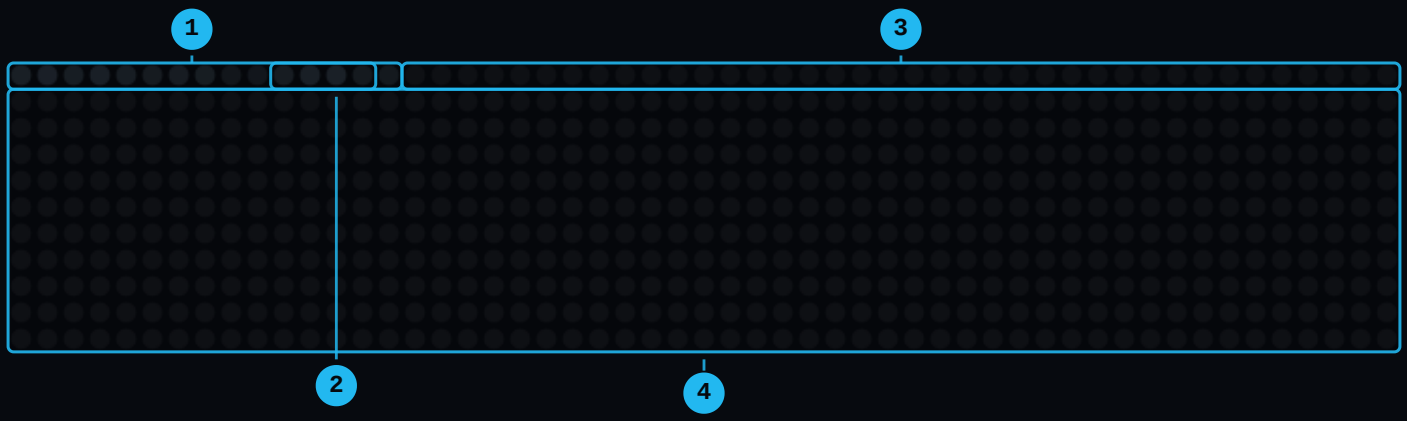
The big 4×7 numerals on theme B are drawn at 30% in slate. They are a background figure to be read at distance, with the small labels painted on top of them.

Text on the matrix is 3×5 pixels. A label drawn at row y occupies rows y through y+4; a big numeral at row y occupies y through y+6. Two consequences follow, and they are why the screens look sparser than they might: a label can start no lower than row 6 without falling off the bottom, and no more than two text elements fit in eleven rows. Screens that tried for three have been cut back to two — the figure you lose was always redundant with one you keep.

SECTION 04 · THEME A · ABSTRACT

A0 · State lattice

The whole roster at once, one LED per system, no glyphs and no labels. The screen you use to see *how much* is wrong, not *what*.



1. **One LED, one system.** Slots fill row-major from the top-left — ordered by tier, then by name — so the core infrastructure always occupies the first cells of row 0.
2. **A gap is not a fault.** Cells with no system behind them stay unlit. On a fleet of fifteen, fourteen rows are legitimately black.
3. **Colour is the state, brightness is the tier.** Slate for operational, amber pulsing for degraded, red for down, violet dashed for maintenance, slate dashed for unknown — each scaled by tier heat, tier 0 brightest.
4. **The whole grid is the canvas.** At 583 slots the lattice holds a fleet far larger than this one; it degrades to a short bright run at small fleet sizes, which is what a healthy akoria looks like.

Reading it

What is normal

A short run of dim, almost-still cells at the top-left and nothing else lit. Individual cells drift in brightness by a percent or two — that flicker is the proof the screen is live.

What to act on

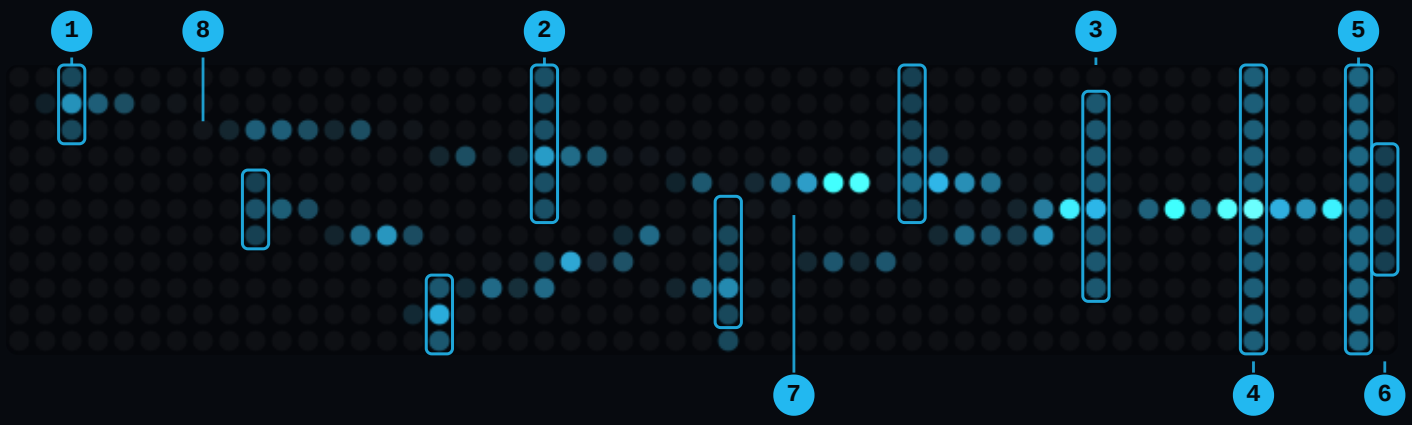
Any red, and especially any *fast-pulsing* red — that is a tier-0 system down. A cell dashing in slate is reporting nothing at all, which on a monitored host is a fault even though it is not red.

A0 does not name anything. It answers "how big is this" in under a second and then sends you to the dashboard. Use B0 for counts and C2 or D1 for which pool.

SECTION 05 · THEME A · ABSTRACT

A1 · Flow gates

The live Ubiquiti path, drawn to scale by role: access points on the left, the internet on the right, traffic flowing between them as particles.



1. **Access points** — 3 rows tall, spread across columns 2–16 and down the height of the panel. Two U7 Pro plus the U7 Pro XGS today.
2. **Hubs** — 5–6 rows, columns 20/27/34, *staggered* top-bottom-top so three of them read as three rather than as a wall.
3. **Switch** — 8 rows at column 41, inset one row top and bottom. Three quarters of the panel height, by role.
4. **Router** — full 11 rows at column 47. The gateway is the only object drawn edge to edge.
5. **Internet** — column 51, full height. It has *no state of its own*: see the rule below.
6. **Backup WAN** — column 52, dimmer and shorter. It keeps its own device state, so during a gateway outage it visibly becomes the only live path.
7. **Legs** — the faint diagonal traces between gates. Their brightness is the source device's *receive* level.
8. **Particles** — travelling left to right along a leg. Spawn rate and brightness follow the source's *transmit* level; a device at level 0 emits none at all.

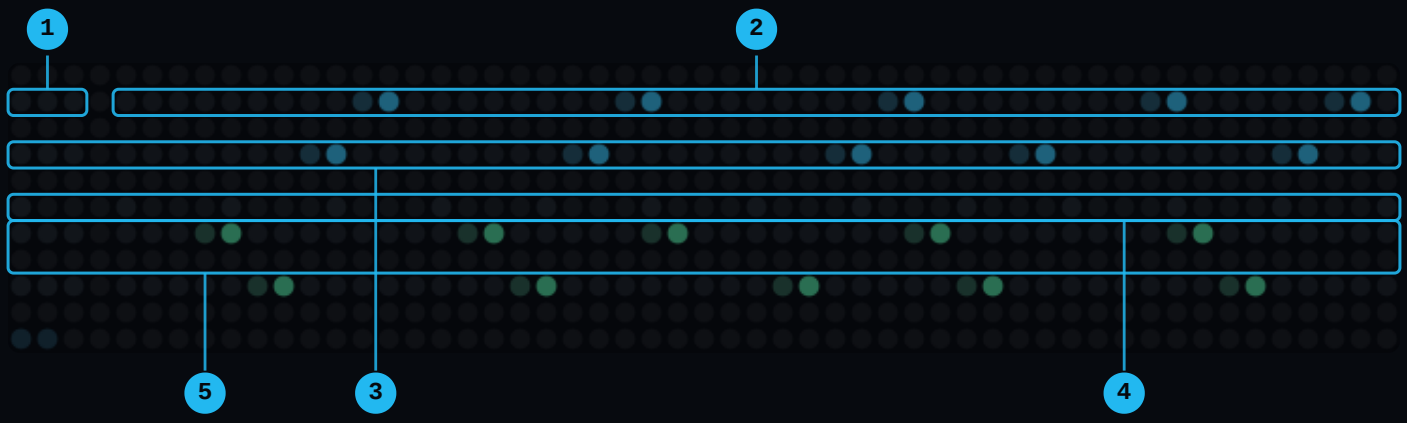
The internet column inherits the router's state. The internet is only reachable *through* the gateway, so when the router gate is degraded or down, column 51 renders in the router's treatment — never a healthy full-height marker over a dead gateway. If no router appears in the payload at all, the column fails dark. The backup WAN beside it is the exception, and that contrast is the story the screen exists to tell.

A down gate is drawn broken. Colour alone does not carry the distinction: a gate in the down state is painted with every other pixel skipped, so it reads as a discontinuity from across the room. Up is azure, degraded is amber, down is red and broken. With no gear data in the snapshot the screen shows `NO DATA` rather than an invented topology.

SECTION 06 · THEME A · ABSTRACT

A2 · MQ and SNMP

Four lanes of moving packets with a backlog indicator on each. Read the name as a shape, not as a promise — what the lanes carry is listed below.



1. **Backlog** — three cells at columns 0–2 per lane. Cells light from the left in proportion to the lane's figure: amber below half, **red above half**.
2. **Trunk and packets** — a dim quiet line from column 4 with five packets travelling it. Packet rate is fixed per lane; it is a liveness animation, not a rate reading.
3. **Row 5** — the dotted divider between the two azure lanes above and the two green lanes below. Structure only.
4. **Packets turn amber** when their lane's backlog passes half, so a loaded lane is legible from the flow as well as from its indicator.
5. **Two azure markers** at the bottom-left corner. Fixed decoration carried over from the design mockup; they carry no reading.

The lanes are not MQ and SNMP. No MQ or SNMP telemetry exists anywhere on the panel's wire protocol. Rather than animate a fiction, the shipped firmware drives the four lanes from the closest real signals the wire does carry. The geometry, rates, colours and thresholds are the original design's; only the inputs are substituted.

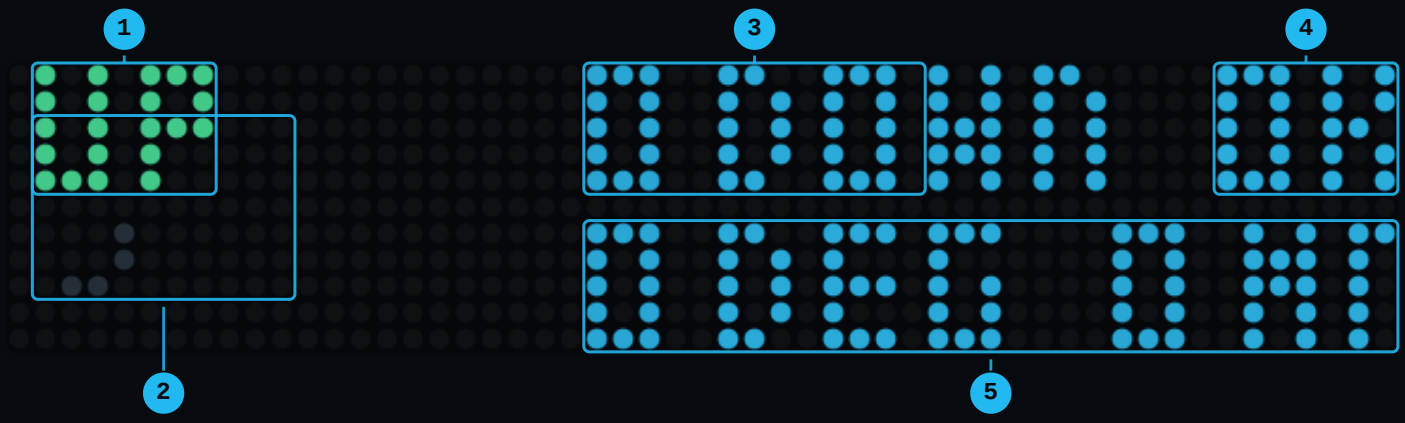
LANE	ROW	LABELLED	ACTUALLY DRIVEN BY
0	1	MQ in	Aggregate probe loss
1	3	MQ out	Host-data freshness — the stale flag, near-empty or near-full
2	6	SNMP poll	Share of the fleet in **unknown** — enrolled, returning nothing
3	8	SNMP trap	Share of the fleet **degraded**

How to use it. Lane 0 backlog in red means probes are losing packets; lane 1 red means monitoring data has gone stale; lanes 2 and 3 red mean a large fraction of the fleet is unknown or degraded. All four clear is the normal condition, and normal looks like four dark indicator blocks with packets sliding past.

SECTION 07 · THEME B · TEXT

B0 · Condition counts

The headline numbers, in words. The one screen that answers "how many, of what" without interpretation.



1. **UP**, in green, at the top-left. The count of agent nodes reporting healthy — the same population as the dashboard headline.
2. **The watermark numeral.** The UP count is also drawn as a large 4×7 figure in dim slate underneath the label. It is faint by design; read it at distance, not up close.
3. **DOWN.** A zero renders azure and very dim; any non-zero count renders **red and bright**. The hue is what distinguishes them, not the digits.
4. **The tier flag word.** `OK` when nothing is down; `TIER 3` when something is down but no tier-0 pool is affected; `TIER 0` in red when it is.
5. **DEG and MNT** on the lower row — degraded and maintenance counts, in amber and violet respectively when non-zero, azure and dim at zero.

Reading it

What is normal

A green `UP` with its watermark, and three azure zeroes. If every figure on the screen is azure, nothing is wrong with the fleet.

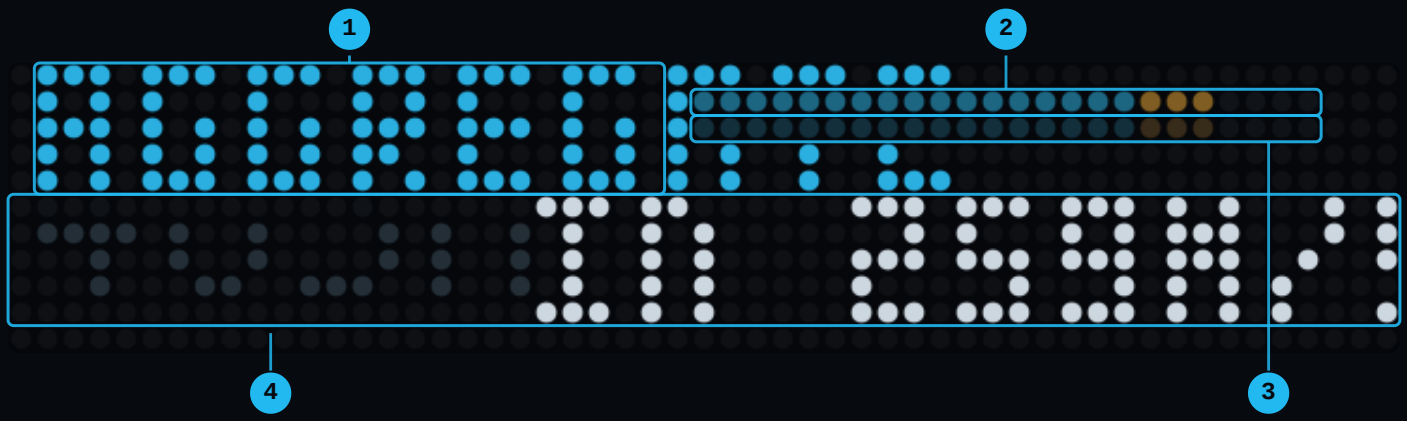
What to act on

`TIER 0` in red is the panel's strongest single statement: core infrastructure is down. It is derived from the server-side pool aggregates over the full fleet, so it cannot be missed by the 64-system wire cap.

The tier flag reads pools, not systems. That is deliberate: the systems array on the wire is capped, and the flag exists precisely to announce the host that a cap might have hidden.

B1 · Throughput

Aggregate rate as a watermark, link utilisation as a 24-cell bar, and a ticker carrying the real ingress/egress split.



1. **AGGREGATE** with the current combined rate as a dim watermark numeral beneath it — the same figure the ticker spells out.
2. **The utilisation bar** — 24 cells, filled left to right by ingress plus egress against the adaptive peak. Cell colour is by *position*: azure below 70%, amber 70–85%, red above.
3. **The echo row** beneath it repeats the bar at a third of the brightness, giving the bar visual weight without doubling its ink.
4. **The ticker**, occupying rows 5–9 and scrolling right to left at 11 px/s: `IN …/S`, `OUT …/S`, `LINK n%`. IN and OUT are the real directions from the wire, not a synthesised split.

The peak is adaptive. The bar is scaled against a running peak the firmware maintains and decays, not against a configured link speed. A quiet period therefore *raises* the apparent utilisation of the next burst. Read the bar as "busy relative to recent history"; read the ticker for absolute figures.

Two readings, at two distances. The **bar** is the at-a-glance figure — you read its length and the colour it has reached from across the room, without reading anything. The **ticker** is the authoritative one, and the only place the absolute rates appear: it carries `LINK n%` alongside the IN and OUT figures that percentage was derived from. There is deliberately no third element. The screen has eleven rows, a five-row label and a five-row ticker already claim ten of them, and every attempt to fit a standalone percent has cost more than it added.

B2 · Load and latency

Mean fleet load as the headline, with latency and loss alternating on a five-second slot rather than fighting for the same row.



1. **LOAD** — the label. It contracts from "MEAN LOAD" because a percent numeral sits directly beneath it and "MEAN" was carrying no information the panel could not spare.
2. **Mean fleet load** as the watermark numeral, in slate — and in **amber above 75%**, which is the only colour change this figure makes.
3. **The alternating reading**, right-aligned to the panel edge, swapping every 5 s: `P95 n MS` (amber above 35 ms) and `LOSS n%` (red above 1%). Both labels stay intact rather than being abbreviated to symbols.
4. **The hot count** — `hot/total>85`: how many systems are running over 85% load, out of how many are on the wire. Amber when non-zero.

The denominator is printed for a reason. The hot count is taken over the panel *roster* — the systems the wire carries, capped at 64 and including adopted probe-only assets — while B0's UP/DOWN are agent-node counts. Printing "3/15" instead of "3" makes it self-evident that this is a different population, so it can never be misread as impossible next to a headline of 9.

Reading it

What is normal

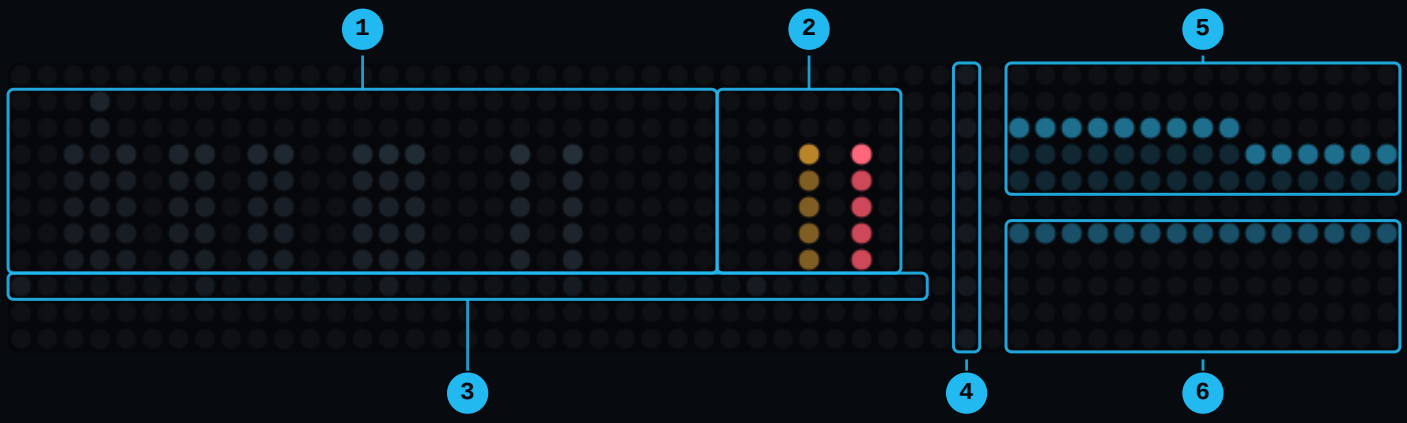
A slate percentage, a white P95 alternating with a white LOSS, and an azure hot count. Watch one full alternation before concluding anything — half the reading is off-screen at any instant.

What to act on

Red `LOSS` is packet loss on the probes and warrants attention before load does. An amber mean load with an azure hot count means the fleet is evenly busy; an azure mean with an amber hot count means a few hosts are pinned.

C0 · Load distribution

Where the fleet's CPU load actually sits, as a 35-bucket histogram, with a short throughput ribbon in the remaining space.



1. **The histogram** — 35 buckets across columns 0–34, left is 0% load and right is 100%. Bar height is the *share* of the fleet in that bucket, rising from row 7, with a brighter tip pixel.
2. **The hot end.** Bucket colour is fixed by the bucket's own load band, not by its height: slate below 75%, **amber 75–90%**, **red above 90%**. A bar in the red zone means systems are actually there.
3. **The axis** — a faint tick every seventh column, i.e. every 20% of load. It is the only scale the screen gives you.
4. **The divider** at column 36, dashed, separating the histogram from the ribbon. Structure only.
5. **Ingress ribbon** — the 15 newest throughput samples, oldest at the left, growing *upward* from row 4. Amber when a sample is above 85% of the adaptive peak.
6. **Egress ribbon** — the same 15 samples growing *downward* from row 6. Always azure; direction is what distinguishes it.

Reading it

What is normal

A dense clump of short slate bars at the left of the histogram and nothing in the amber or red bands. The ribbon breathes as traffic moves.

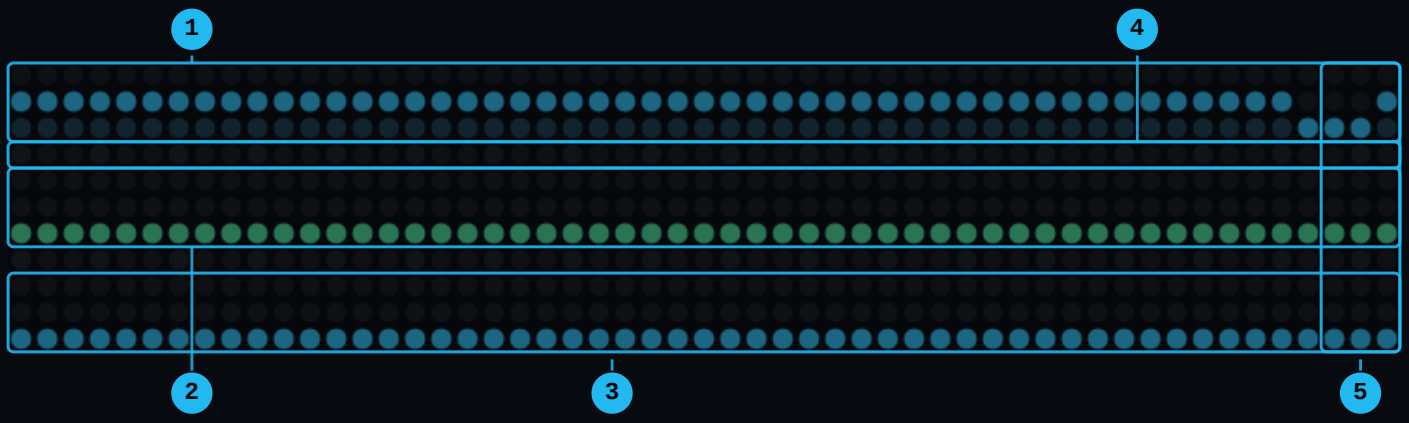
What to act on

Any bar standing in the red band. C0 tells you the shape of the load — a broad clump is an evenly busy fleet, a spike at the right is a small number of pinned hosts. B2's hot count gives you how many; C2 and D1 give you which pool.

SECTION 11 · THEME C · CHARTS

C1 · Live traces

Three stacked histories across the full width of the matrix. The only screen that shows you a trend rather than a moment.



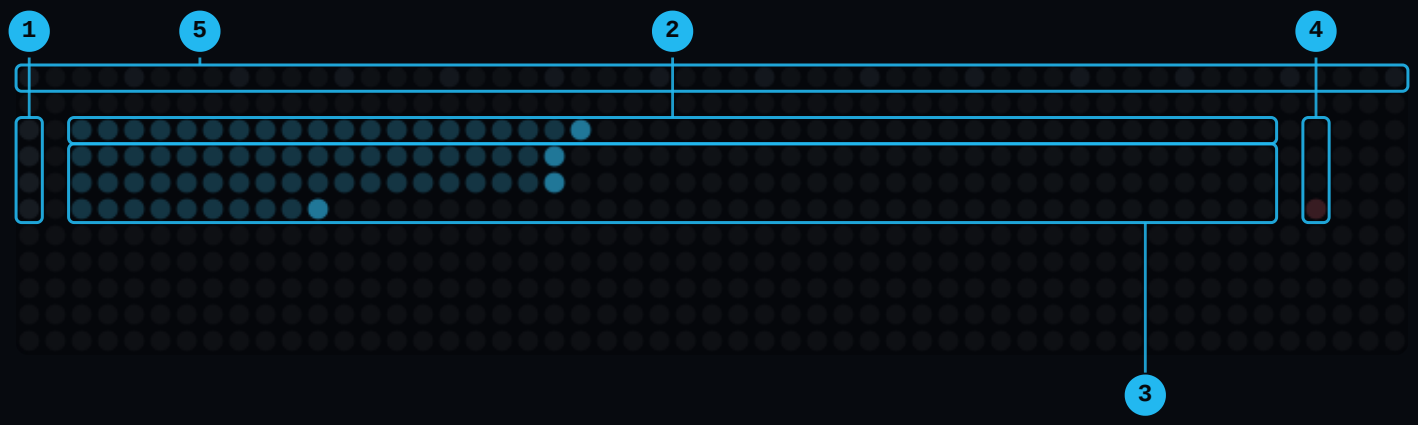
1. **Throughput**, rows 0–2, azure — turning **amber** for any sample above 85% of the adaptive peak.
2. **CPU**, rows 4–6, green — amber above 80%. Green here is a trace colour for a healthy metric, and the only green band on the screen.
3. **Round-trip time**, rows 8–10, azure — amber above 70% of full scale. Latency and throughput share a hue; their position is what tells them apart.
4. **Separator dots** on rows 3 and 7, every third column. Structure only — they keep three 3-row traces from reading as one 11-row texture.
5. **Now is the right edge.** Each trace is 53 samples, oldest at column 0. The traces advance one column per snapshot; they do not scroll continuously.

Every trace has a floor of one pixel. A row of single dim pixels is a real zero reading, not a dead screen. The tip pixel of each bar is painted brighter than its body, so the profile of a trace is legible even when every bar is one or two pixels tall.

History is the panel's, and it is lost on reboot. Nothing on this screen comes from the server. After a firmware restart the traces refill from the left over about two minutes at the 2 s snapshot rate. A partially filled trace is a recently restarted panel, not a fleet that just came online.

C2 · Pool load

Up to seven pools, worst first, each with a status key and a wide load bar. The screen that answers "which pool".



1. **The status key**, column 0 — one pixel per pool, coloured by its *worst* member: down beats degraded beats unknown beats maintenance beats healthy.
2. **The load bar** — 46 cells across columns 2–47, the pool's mean load. Cell colour is by position: azure to 70%, amber to 85%, red beyond.
3. **Worst-first ordering.** Row 2 is the pool in the most trouble, not the first pool alphabetically. With only seven rows visible, this is what keeps a failing pool on the screen.
4. **Tier-0 marker** at column 49 — a dim red pixel beside any tier-0 pool, whether or not it is in trouble. It marks importance, not condition.
5. **Row 0 ticks**, every fourth column, very faint. A scale for the bars beneath, and the only thing on the screen that is not a pool.

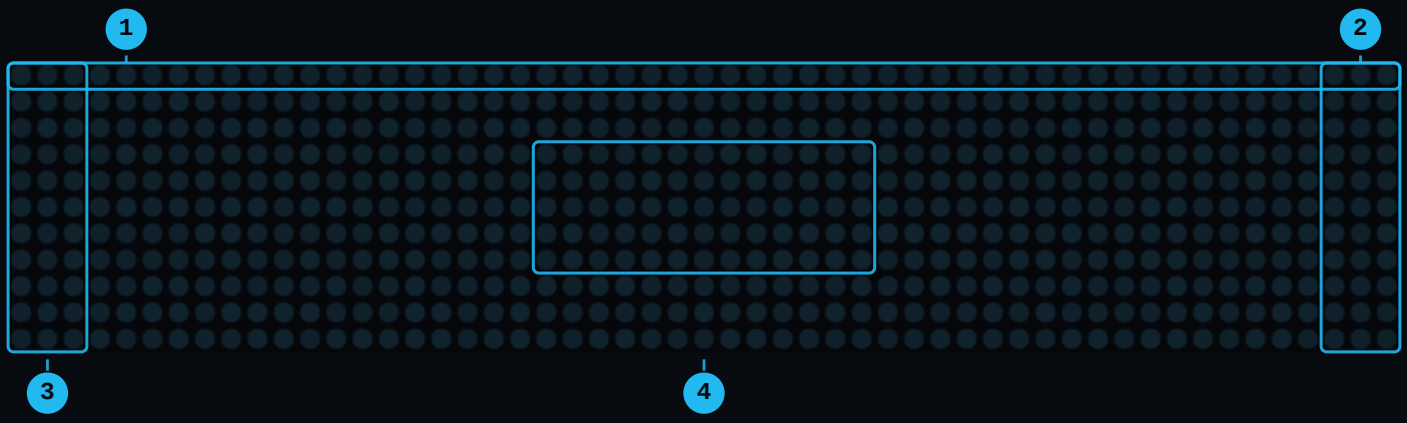
The screen carries no title. Pool rows occupy rows 2–8, and the shape — a key pixel, a wide bar, repeated — identifies it without one.

The key and the bar answer different questions. A pool can be entirely healthy and heavily loaded (slate key, long red bar) or idle and broken (red key, short azure bar). Read the key column first — it is the condition — then the bar for capacity.

Severity order is fixed at down > degraded > unknown > maintenance. Unknown outranks maintenance deliberately: a pool nobody can reach is a worse situation than one that is down for planned work. D1 uses the same order, so the two pool screens can never disagree.

D0 · Reachability

A waterfall: eleven rows of probe history scrolling right to left, one new column every 0.42 s. The screen that shows you a fault's *shape over time*.



1. **One row per system** — the eleven worst-first systems on the roster. Row 0 is the one in the most trouble. The rows are not named; use C2 or the dashboard for identity.
2. **Now is the right edge.** A new sample column is pushed every 0.42 s and everything shifts left, so the full width is about 22 s of history.
3. **Oldest is the left edge.** A fault that has just started is a red block hugging the right; one that is recovering is a red block drifting left with clean cells behind it.
4. **Cell colour is the verdict. Red** at full brightness for a lost probe, **amber** for slow, otherwise azure whose brightness carries the round-trip time — brighter is slower, on a 100 ms full scale.

Reading it

What is normal

A uniform dim azure field with visible texture — the per-cell jitter is intentional, and it is how you tell a live waterfall from a frozen one. Brightening rows mean latency is rising fleet-wide.

What to act on

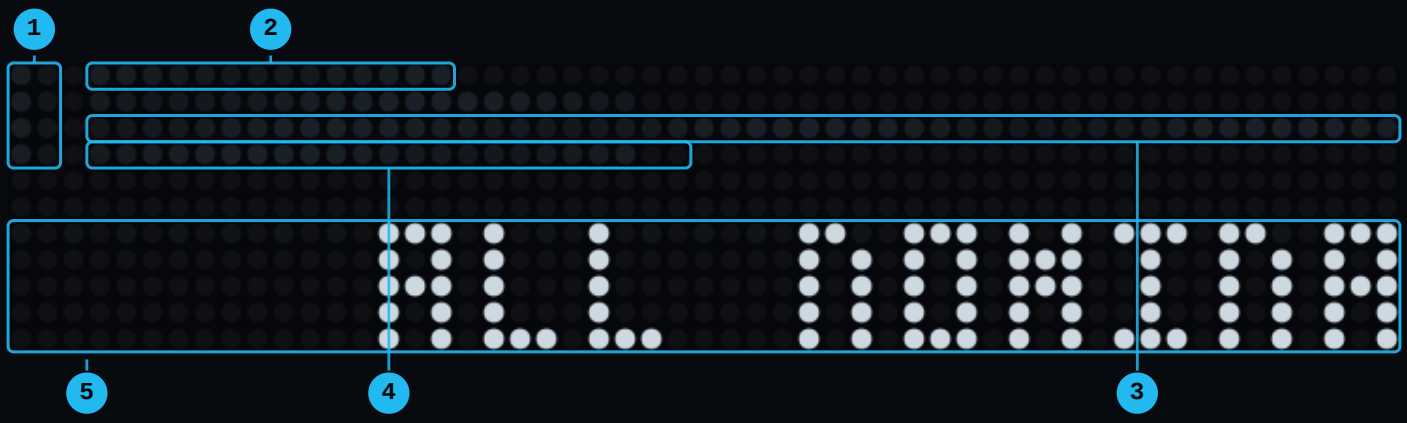
Solid red rows: that system is down. Speckled red across many rows: probe loss is affecting everything, which points at the network rather than at a host. Amber-dominant rows are degraded or unknown systems.

The rows are systems, not named probe targets. The wire carries fleet-aggregate latency and loss only, so each row samples one worst-first system against those aggregates: a down system samples mostly as loss, a degraded or unknown one mostly as slow, and everything else samples healthy with the aggregate RTT jittered per column. The texture is real; the per-row precision is not. Treat D0 as a shape, and confirm identity elsewhere.

SECTION 14 · THEME D · INSTRUMENTS

D1 · Pool bars

The resting face. Every pool drawn proportional to its size, broken down by condition, with the live alert ticker underneath.



1. **The status key**, columns 0–1 — two pixels per pool, coloured by its worst member on the same severity order C2 uses.
2. **Bar length is pool size**, scaled against the largest pool, with a six-pixel minimum so a two-host pool is still visible. It is *not* load — that is C2's bar.
3. **Stacked from the right edge**: down, then degraded, then unknown, then maintenance, each at least one pixel if the pool has any. Worst conditions therefore always terminate the bar.
4. **The idle shimmer** fills the rest — the healthy remainder of the pool, breathing slowly. Motion here is the screen's liveness signal.
5. **The alert ticker**, scrolling at 13 px/s over a dotted line on row 6. This is where a fault gets named — the only place on the panel, outside an alarm, that carries text about a specific alert.

Reading it

What is normal

Slate keys, bars that are entirely shimmer, and a ticker with nothing urgent in it. Up to six pools fit; worst-first ordering means the ones you cannot see are the ones that are fine.

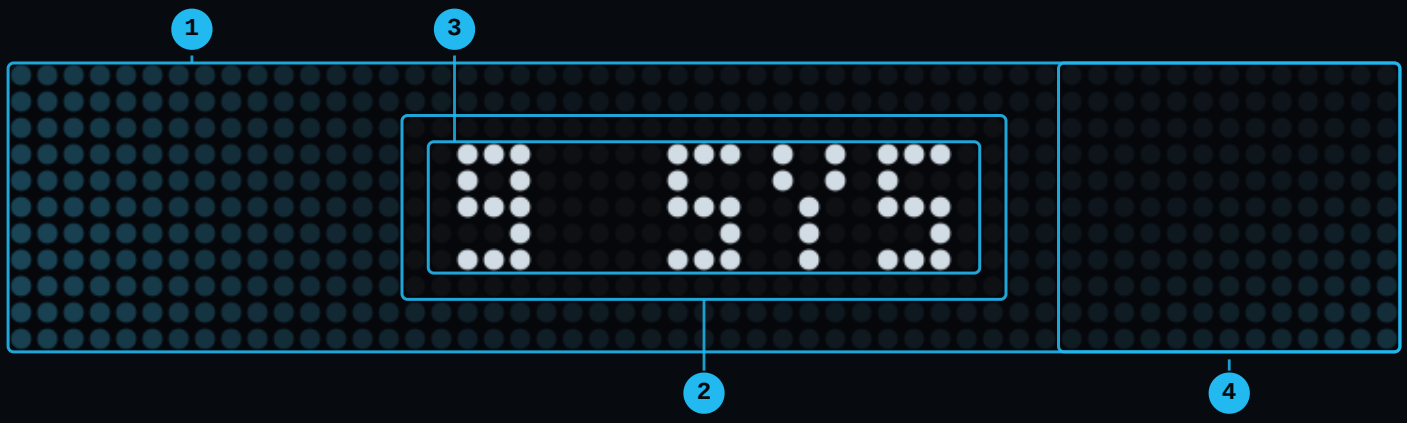
What to act on

Any coloured segment at the right end of a bar, and its length relative to the bar: a two-pixel red tail on a long bar is one host in a large pool; a bar that is half red is a pool in serious trouble.

Why the unknown band exists. Hosts returning no telemetry used to fall into the idle shimmer and read as healthy. They now have their own slate band in the stack, between degraded and maintenance, so a pool full of silent hosts cannot look like a pool full of well ones.

D2 · Ambient field

A procedural field whose colour is the fleet's mean load and whose sparks are its failures, with a three-slot readout in the middle.



1. **The field** — layered noise across the whole matrix. Its *granularity* grows with fleet size and its **hue is mean load**: azure when the fleet is quiet, warming toward amber from 35% load and fully warm by 85%.
2. **The knockout box** — a dimmed rectangle behind the readout, so text stays legible over a moving field without stopping the field.
3. **The readout**, cycling every 5 s through three slots: `n SYS` (fleet size), `n% LOAD` (amber when the field is hot), and the aggregate rate per second.
4. **Sparks** — up to nine red pixels with a soft horizontal bloom, one per down system, pulsing at a rate set by the tier of the first down system found. On a healthy fleet there are none.

Reading it

What is normal

A cool, slowly drifting azure field with a readout in the middle and no red anywhere. This is the screen the panel is most often left on; it is meant to be pleasant and to change colour before you consciously notice.

What to act on

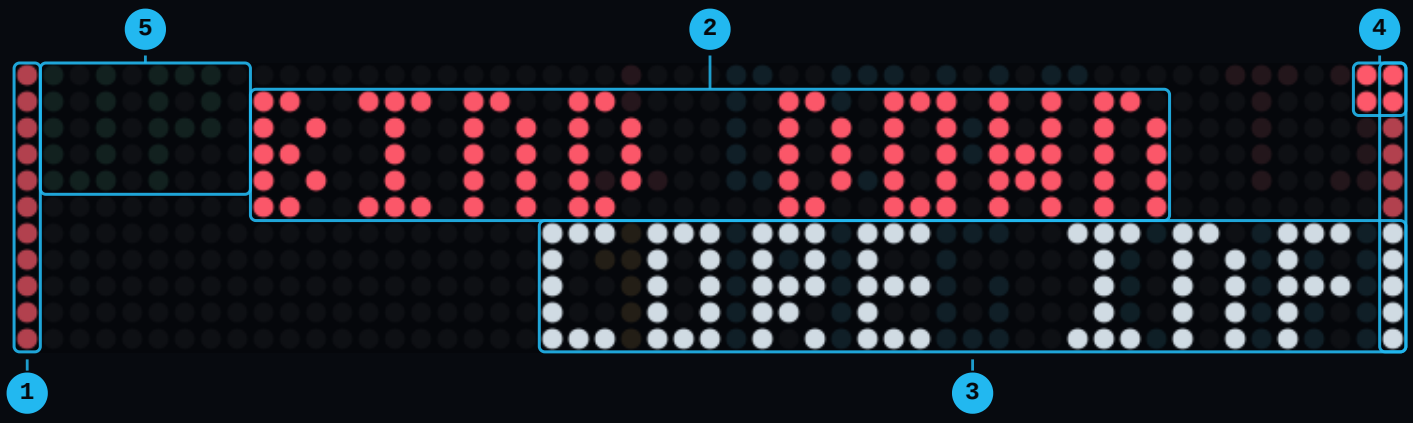
Red sparks — each one is a system down — and a field that has gone amber, which means fleet-wide load has climbed past the two-thirds mark. Nine sparks means "nine or more"; go to B0 for the true count.

The readout counts agent nodes. `n SYS` is the same population as B0's UP/DOWN — the dashboard headline — not the wider roster that A0's lattice and the pool screens draw from.

SECTION 16

Alarms

One overlay, one beacon, one tone, and an acknowledge that silences without clearing. Everything on this page overrides whatever screen is beneath it.



1. **Edge rails** — columns 0 and 52, full height, red, pulsing about three times a second. The unmistakable part.
2. **Subject**, centred on row 1, red and bright. Truncated to twelve characters — it names the fault class, not the whole story.
3. **Detail**, scrolling on row 6 at 11 px/s in white. Composed server-side, e.g. `RESTART BIND ON MOLYBDENUM`.
4. **The beacon** — four pixels at the top-right corner, flashing red at 1 Hz. Painted last, over everything, including the rail.
5. **The screen beneath** is dimmed to 10%, not replaced. Rotation continues underneath; the inlay follows it.

The tone

A three-note square triad — 990 Hz, 660 Hz, 990 Hz, 200 ms each, at 0, 0.22 and 0.44 s — sounded immediately on the rising edge and then **every 60 s** until acknowledged. VOL ± sets its level.

Auto-silence

An episode left unacknowledged for **5 minutes** stops re-sounding, permanently, for that episode. Auto-silence is *not* an acknowledge: the inlay stays, the beacon keeps flashing, and a new episode re-arms the tone as normal.

Acknowledge silences; it never clears. The alert remains active on the dashboard and in the alerting pipeline. Acknowledging is a statement that a human has seen the panel, nothing more.

Ack is scoped to the episode. The server composes an episode id for each alarm episode; acknowledging records that id on the panel. A *new* episode id re-arms everything — inlay, tone and beacon — even if the previous one was acknowledged seconds earlier. There are exactly two ways to acknowledge: any physical button while the alarm is armed, or the control page's ACK button. Both run the same code path, so they cannot drift apart.

The rising edge wakes a sleeping panel once — and only once. It does not hold the panel awake. If nobody acknowledges, the display sleeps again on its own schedule and the beacon is what carries the fault through the night. That is the design, and it is why the beacon is exempt from dimming.

SECTION 16 · CONTINUED

The alarm over each theme

The inlay is the same on all twelve screens. What differs is what survives underneath it at 10% — which is the point: you can still tell what the panel was showing when the alarm arrived.



A1 · the gate columns remain faintly visible through the inlay.



B2 · theme B's watermark numerals dim to nearly nothing.



C1 · the three trace bands stay readable as texture.



D2 · the field survives dimming better than any other screen.

What is constant

- **Both edge rails**, full height, pulsing red.
- **The subject** on row 1, twelve characters, centred.
- **The detail** scrolling on row 6.
- **The beacon**, four pixels top-right, 1 Hz, painted over everything including the rail.
- **The 10% dim** applied to whatever screen is beneath.

What acknowledging changes

- The **tone stops**, permanently, for this episode.
- The **beacon stops** flashing.
- The **inlay remains** — rails, subject and detail — because the fault has not gone anywhere.
- A **new episode id** re-arms all three from scratch.

An acknowledged alarm still looks like an alarm. That is intentional. The quiet, unblinking version of this overlay means "a human has seen this and it is still broken" — not "handled". It clears when the underlying alert clears, and by no other route.

SECTION 17

The control page

The dashboard's **Panel** page: all twelve screens simulated live, plus every control the physical buttons offer and several they do not.

CONTROL	EFFECT	NOTES
Theme A–D	Jump to a theme	Identical to pressing that theme's button on the panel
Screen 0–2	Jump to a slot	Accepted even if the slot is disabled — operator intent wins; rotation resumes per configuration at the next advance
Brightness	Set a manual level	Latches manual exactly like LUX ±. **AUTO** is the only way back to sensor control. Floors at 25%
Acknowledge	Silence the alarm	Disabled unless an episode is armed. Does not clear the alert
Dwell	3 / 6 / 30 s	The *base* interval each screen's weight multiplies. 6 s is the boot default
Sleep / Wake	Blank or restore	Explicit. Other commands deliberately do *not* wake a sleeping panel
Rotation On/Off	Per screen	On the screen's own tile. A skipped screen is greyed and badged
Dwell weight	¼× ... 10×	Also per tile. See section 18

Pending, applied, failed

A control on the page is not a switch that sets a value; it is a request that the panel may confirm. Understanding the three states is the difference between trusting the page and second-guessing it.

Pending

The command is queued and being re-served to the panel every poll. **Nothing is shown as selected** while a change is in flight — the old value is withheld rather than left standing, so it cannot be misread as the new one having landed.

Applied

The panel reported back that it consumed the command. Only now does the page mark an option as in force.

Failed

Two minutes elapsed with no confirmation, so the command expired. The page says so, with a reason. It is never silently dropped.

The panel is the authority, always. A selected chip on the control page means *the panel said so* — never that the page sent something. Before the panel has reported at all, nothing is selected and the page says so in words rather than showing a cheerful default. This is why a control can appear to "do nothing" for a few seconds: latency is one poll cycle plus a snapshot interval, roughly 5–7 s, and the page will not pretend otherwise.

Read-only by default

Controls require the operator or admin role. A viewer, an unconfirmed role, or a failed role lookup all render the page read-only — it fails closed, and the server enforces this independently of what the page shows.

The simulation is an approximation

The twelve virtual screens resample the 5 s dashboard poll onto the same 53-sample rings the firmware fills at 2 s. Ribbons, histograms and traces will differ in detail from the wall even when the fleet data is identical.

SECTION 18

Rotation & weights

Which screens the panel cycles through, and for how long each. Set per screen, stored on the panel itself, and confirmed only by the panel.

How long a screen stays up

Every screen has a weight that multiplies the base dwell. The base is a single global setting — 3, 6 or 30 s — and the weight is per screen.

¼×	½×	1×	2×	5×	10×
7.5 s	15 s	30 s	60 s	150 s	300 s

Dwell at a 30 s base. So with the base at 30 s, a screen at 5× holds for 150 s, one at ½× for 15 s, and one at 1× for the base 30 s — which is the usual way to build a rotation that lingers on the screen you care about and passes quickly over the rest.

Turning a screen off

A disabled screen is skipped by the rotation and by manual advance. Its tile on the control page greys out and is badged `SKIPPED`. Disabling is not deletion — the screen still renders in the simulation, and re-enabling takes effect at the next advance.

Changing a weight mid-dwell

Elapsed time is *rescaled*, not reset: a screen 299 s into a 10× dwell that is changed to ¼× advances immediately rather than holding for another seven seconds. Shortening a weight always shortens the wait.

You cannot blank the panel this way. If every screen of the active theme is disabled, the firmware treats the theme as fully enabled rather than showing a black wall. The same guard applies to a jump command targeting a disabled slot: it is accepted and displayed, and rotation resumes per the configuration at the next advance.

Where the configuration lives

On the panel — written into the last sector of the RP2350's flash, and reloaded at boot. It survives resets, power cuts and a change of network, and it is not a dashboard setting that gets pushed down at startup. A corrupt or absent record falls back to the factory default (every screen enabled at 1×) rather than failing.

WRITE POLICY	`on change only, debounced 2 s`
FACTORY DEFAULT	`all 12 enabled · weight 1×`
CONFIRMATION	`page shows “Saved to panel flash” on a positive report`

Read the page's chips, not your memory. The panel reports its own configuration back, and the page renders *that*. A summary chip near the top reads, for example, `10 of 12 screens in rotation · 2 skipped · 3 weighted`, and screens the panel has not reported are counted separately as unreported rather than folded into "on". If the chip says `Screen config not reported`, the panel has not told the dashboard anything yet — the settings you last chose may well be in force, but nothing on the page is evidence of it.

SECTION 19

The glance matrix

lithium — the UNO Q hosting the panel — has a 13×8 monochrome matrix of its own. It watches the panel's own chain, which the panel cannot do for itself.



The glance is a grayscale frame, redrawn four times a second from facts the panel daemon publishes every two seconds. It is a health readout for the *panel chain*, not a second fleet display — it is what tells you whether the wall is telling the truth.

MATRIX	13 × 8 · 8 grey levels
FACT REFRESH	1 Hz read · 15 s staleness
ANIMATION	0.25 s tick

REGION	WHERE	MEANING
Poll dot	row 0, col 0	Steady: the daemon is reaching the API. **Blinking bright: it is not.**
Serial dot	row 0, col 2	Steady: the USB link to the panel is up. Blinking: it is down
STATE dot	row 0, col 4	Bright once the panel has reported state — i.e. the firmware speaks the control protocol. Dim means commands are being withheld
Alarm dot	row 0, col 12	Flashing: armed and unacknowledged. Steady mid-grey: acknowledged
Score bar	rows 2–3	The fleet score, 0–200 mapped across the full width. Longer is worse
Heartbeat	rows 2–3, centre	A single pixel breathing on a ~3 s cycle when the score is zero — a calm fleet still has to read as alive
Fleet strips	rows 5, 6, 7	Up (bright), degraded (mid), down (blinking) — one pixel per system, capped at 13

Two whole-matrix behaviours. An armed, unacknowledged alarm pulses the entire glance, mirroring the wall beacon. A sleeping panel halves every level, so the glance visibly dims with the panel it reports on. And if the facts file is missing or older than 15 s, everything else goes dark and a **single pixel blinks at the top centre** — the feed is alive but the daemon is not publishing.

SECTION 20

Troubleshooting

Four things the panel shows when the problem is the panel, and how to tell them from a fleet fault.



Reference — a healthy panel is mostly dark, but never still.

Nothing is lit at all

Check in this order: the panel may be **asleep** (any button wakes it — but look for the beacon first, four flashing red pixels top-right, which means it is asleep *with a fault on it*); brightness may be at its floor; or the firmware may not be running. The glance matrix on lithium settles this in one look: if its serial dot is blinking, the panel is not being spoken to.

WHAT YOU SEE	WHAT IT MEANS	WHAT TO DO
LINK LOST	No valid frame from the daemon for over 15 s	A **panel** fault, not a fleet fault — hence amber, no rails and no tone. Check `solari-panel` on lithium and the USB cable. A single pixel sweeps a row underneath the message, so a frozen render is distinguishable from a live one
NO DATA	A zero-node fleet, or no snapshot has arrived yet	Normal for a few seconds after a panel restart. Persisting means the API is returning an empty fleet — check the dashboard, not the panel. Azure, because it is a valid reading rather than an error
NO DATA on A1 only	No network-gear section in the snapshot	The UniFi poller is not delivering, or its rows have aged past 60 s. Every other screen stays correct. A1 shows nothing rather than an invented topology
Screens look right but **stale**	Monitoring data older than 30 s	The panel keeps rendering the last good state on purpose. The control page shows an `Upstream data stale` chip; A2's lane 1 backlog also lights. Look upstream of the panel entirely

When the wall and the page disagree

Detail differs

Expected. The simulation resamples a 5 s poll onto rings the firmware fills every 2 s, so traces, ribbons and histograms will not match pixel for pixel even on identical data. Counts, states and pool bars should match.

State differs

Check the page's `Panel state … old` chip. Beyond about 90 s the page is showing a stale report, not the current panel. The panel is the authority; the page is its most recent account of itself.

A last cross-check. Three things must agree before you trust a reading: the panel, the control page, and the dashboard's own alert count. All three are derived from the same dashboard-active alert definition, so a genuine disagreement between them is a bug worth reporting rather than a number worth averaging.

SECTION 21

Standalone WiFi

The same panel, the same screens, no lithium in the chain: a provisioned panel speaks the identical frame protocol over mutually-authenticated TLS to the daemon on xenon.

How it connects

At boot the panel first fetches the time (certificates cannot be judged without a clock — TLS is not attempted until the clock is plausible), then joins the provisioned network and connects to `panelgw.akoria.net:7443`. Panel and daemon verify each other's certificates against the same private CA. Live fleet state appears within one poll cadence of the link coming up.

When it drops

AP restart, DHCP renew, daemon restart — all the same case: the panel backs off and reconnects on its own, starting at 1 s and capping at 60 s between attempts. Nothing to do at the wall. Meanwhile the last good state keeps rendering under the ordinary staleness rules.

USB always wins

Plug the panel into any host running the daemon and it switches to USB within 15 s of the first valid frame, suspending WiFi. Unplug, and WiFi resumes (an unplug is detected and skips the wait). That is the whole bench workflow: walk up, plug in, work, unplug.

Unprovisioned = today's panel

With no stored credentials the WiFi transport stays inactive — no association, no network traffic — and USB serial behaves exactly as before standalone mode existed. (The CYW43 chip itself is still initialised at boot: it also senses VBUS for the power alarm.) Credentials are written only over USB with the `panelProv` bench tool — the firmware refuses provisioning frames on WiFi, permanently.

ON THE WALL	WHAT IT MEANS IN STANDALONE MODE
LINK LOST	Unchanged meaning: no fresh snapshot past the staleness bound, on *any* transport. WiFi outage, daemon down on xenon, or certificate trouble all end up here once the last snapshot ages out
Long LINK LOST after a power cut	The panel may be waiting on time sync before it will attempt TLS — it retries on its own. If it persists, check the xenon journal: daemon-side refusals are logged as `TLS client refused … reason=…`. (The reverse case — the panel rejecting the daemon's certificate — shows only as a failed connection on the daemon side, with the detail on the panel's USB debug console.)
Fine on USB, dead on WiFi	Credentials, certificates, or the listener — not the panel's renderer. Start at `journalctl -u solari-panel` on xenon; the standalone runbook's inventory page lists what should be valid

A clean switch never blips the wall. Transport changes with fresh snapshots in hand do not trigger LINK LOST — that message is bound to data staleness only. If you see it during a USB plug/unplug, something else is actually stale.

Operations — certificate issuance, provisioning, renewal, revocation — live in the companion document `SolariNet_Panel_Standalone_Runbook.html`. This manual covers only what the wall shows.

SECTION 22

Power & battery

The battery is a ride-through UPS, not a portable mode. Losing wall power is itself an alarm — the panel dims hard, marks itself, and sounds.

When USB power disappears (debounced over 100 ms, so a flicker does not count), three things happen at once:

1 · Brightness caps

Output clamps to the battery ceiling (build default 20%, field-calibratable). LUX± still work underneath the cap — the panel remembers what you asked for and returns to it exactly when power is restored.

2 · The glyph

A small steady amber battery outline appears at the top right (columns 49–52, rows 6–10), with the pixels behind it knocked back so it reads at a glance. It shows the entire time the panel is on battery — through help pages, through sleep, alarm or no alarm — and never touches the beacon's corner.

3 · The alarm arms

Power loss raises its own alarm episode, with the usual tone cadence, beacon and night-profile rules. Any button acknowledges it — and an acknowledge covers *all* current noise, server alerts included.

SITUATION	BEHAVIOUR
Power restored	Cap lifts to your previous brightness, glyph disappears, the power alarm clears itself. A live server alert is untouched — restore never silences the fleet
Second outage, same day	A fresh episode: the panel alarms again. Acknowledging the first outage never mutes the next one
Server alert while on battery	The inlay text shows the fleet alert — the server always wins the words; the glyph carries the power story. One button press acknowledges both
Boot on battery	Treated as a power loss from the first instant: capped, glyph on, alarm armed — before any data has even arrived
Battery runs out	The pack's protection cuts off and the panel dies cleanly. It resumes on wall power or USB with no ceremony — there is no low-battery warning by design

Measured, not promised. Ride-through runtime at capped brightness with WiFi associated: `[OPERATOR: from the D19 measurement session]`. The pack itself — capacity, protection cutoff, connector polarity checked against the silkscreen — is recorded in the P4 design addendum *before* it is ever connected. The board has no charger: the battery does not recharge from USB.

Why the panel alarms about itself. A panel on battery is a room that lost power — exactly the kind of event the fleet exists to notice. The alarm is not the panel complaining; it is the panel doing its job with the last power it has.